## FINAL Unit1-Unit2 FRQ - Topic order

This question involves the RaceCar and SimulateCarRace classes, which are used to simulate a race between two race cars. Each race consists of one or more laps around a racetrack, starting with lap 1. Each lap consists of one complete trip around the racetrack.

The RaceCar class contains a helper method, getLapTime. You will write two methods in the SimulateCarRace class.

```
public class RaceCar
{
  /**
   *  Returns the number of seconds it takes the race car
   *  to complete the lap specified by the parameter
   */
  public double getLapTime(int lapNum)
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

```
public class SimulateCarRace
{
  private RaceCar carOne;
  private RaceCar carTwo;

  /**
   *  Simulates a race with numLaps laps and returns a String
   *  indicating the outcome, as described in part (a)
   *  Precondition: numLaps > 0
   */
  public String findWinner(int numLaps)
  { /* to be implemented in part (a) */}

  /**
   *  Returns a shortened version of the parameter message with all
   *  substrings that are equal to the parameter str removed, as
   *  described in part (b)
   *  Precondition: There are no overlapping occurrences of str
   *                within message.
   */
  public String shortenMessage(String message, String str)
  { /* to be implemented in part (b) */}
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

**1.** Write the findWinner method, which simulates a race between two cars consisting of numLaps laps around a racetrack. Each car takes a different amount of time to complete each lap, which can be found using the getLapTime method in the RaceCar class. For example, calling getLapTime(3) on a RaceCar object will return the number of seconds it takes the car to complete lap 3 of the race.

The findWinner method returns a String indicating the outcome of the race, according to the following rules.

- "Car 1 wins!" is returned if carOne completes the race in fewer seconds than carTwo.
- "Car 2 wins!" is returned if carTwo completes the race in fewer seconds than carOne.
- "Tie!" is returned if the cars complete the race in the same amount of time.

Complete method findWinner. You must use getLapTime appropriately in order to receive full credit.

```
/**
 * Simulates a race with numLaps laps and returns a String
 * indicating the outcome, as described in part (a)
 * Precondition: numLaps > 0
 */
public String findWinner(int numLaps)
```

### Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getLapTime` for at least one of `carOne` or `carTwo`

**Canonical Solution:**

```
public String findWinner(int numLaps)
{
   double car1Time = 0;
   double car2Time = 0;
   for(int j = 1; j <= numLaps; j++)
   {
      car1Time += carOne.getLapTime(j);
      car2Time += carTwo.getLapTime(j);
   }
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   if (car1Time < car2Time)
   {
      return "Car 1 Wins!";
   }
   else if(car2Time < car1Time)
   {
      return "Car 2 Wins!";
   }
   else
   {
      return "Tie!";
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to `getLapTime`

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loops `numLaps` times

**Canonical Solution:**

```
public String findWinner(int numLaps)
{
   double car1Time = 0;
   double car2Time = 0;
   for(int j = 1; j <= numLaps; j++)
   {
      car1Time += carOne.getLapTime(j);
      car2Time += carTwo.getLapTime(j);
   }
   if (car1Time < car2Time)
   {
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
      return "Car 1 Wins!";
   }
   else if(car2Time < car1Time)
   {
      return "Car 2 Wins!";
   }
   else
   {
      return "Tie!";
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

### Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calculates the total amount of time for at least one race car

**Canonical Solution:**

```
public String findWinner(int numLaps)
{
   double car1Time = 0;
   double car2Time = 0;
   for(int j = 1; j <= numLaps; j++)
   {
      car1Time += carOne.getLapTime(j);
      car2Time += carTwo.getLapTime(j);
   }
   if (car1Time < car2Time)
   {
      return "Car 1 Wins!";
   }
   else if(car2Time < car1Time)
   {
      return "Car 2 Wins!";
   }
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   else
   {
      return "Tie!";
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to use the total

Responses will not earn the point if they

- fail to initialize a variable for the total
- accumulate times for both cars into the same variable

### Part A Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns a correct string based on the calculated total time for each race car (*algorithm*)

**Canonical Solution:**

```
public String findWinner(int numLaps)
{
   double car1Time = 0;
   double car2Time = 0;
   for(int j = 1; j <= numLaps; j++)
   {
      car1Time += carOne.getLapTime(j);
      car2Time += carTwo.getLapTime(j);
   }
   if (car1Time < car2Time)
   {
      return "Car 1 Wins!";
   }
   else if(car2Time < car1Time)
   {
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
      return "Car 2 Wins!";
   }
   else
   {
      return "Tie!";
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- fail to compare lap times
- fail to return the string
- print the string instead of or in addition to returning it

## FINAL Unit1-Unit2 FRQ - Topic order

**2.** Write the shortenMessage method, which returns a shortened version of the parameter message with all substrings that are equal to the parameter str removed. If str does not appear in message, the original value of message is returned.

For example, shortenMessage("the_driver_of_the_first_car_is_Jerry", "the_") should return "driver_of_first_car_is_Jerry".

Complete method shortenMessage.

```
/**
 * Returns a shortened version of the parameter message with all
 * substrings that are equal to the parameter str removed, as
 * described in part (b)
 * Precondition: There are no overlapping occurrences of str
 *               within message.
 */
public String shortenMessage(String message, String str)
```

---

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loop accesses all necessary parts of message string (*no bounds error*)

**Canonical Solution:**

```
public String shortenMessage(String message, String str)
{
   String result = message;
   while (result.indexOf(str) >= 0)
   {
      int idx = result.indexOf(str);
      result = result.substring(0, idx) +
               result.substring(idx + str.length());
   }
   return result;
}
```



| 0 | 1 |
|---|---|

## FINAL Unit1-Unit2 FRQ - Topic order

The response earns the point.

Responses will not earn the point if they

- use either a condition or an update that would cause an infinite loop or skipped elements in some cases

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies a location of `str` within `message` if there is one (and all `String` method calls are syntactically valid)

**Canonical Solution:**

```
public String shortenMessage(String message, String str)
{
   String result = message;
   while (result.indexOf(str) >= 0)
   {
      int idx = result.indexOf(str);
      result = result.substring(0, idx) +
               result.substring(idx + str.length());
   }
   return result;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to use the identified location

### Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Builds and returns the correct string derived from first parameter with substring removed as many times as necessary (*algorithm*)

## FINAL Unit1-Unit2 FRQ - Topic order

**Canonical Solution:**

```
public String shortenMessage(String message, String str)
{
   String result = message;
   while (result.indexOf(str) >= 0)
   {
      int idx = result.indexOf(str);
      result = result.substring(0, idx) +
               result.substring(idx + str.length());
   }
   return result;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- fail to return the string
- print the string instead of or in addition to returning it

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves the MessageBuilder class, which is used to generate a message based on a starting word. The MessageBuilder class contains a helper method, getNextWord. You will write a constructor and a method in the MessageBuilder class.

```
public class MessageBuilder
{
  private String message;    // To be initialized in part (a)
  private int numWords;      // To be initialized in part (a)

  /**
   *  Builds a message starting with the word specified by the parameter and
   *  counts the number of words in the message, as described in part (a)
   *  Precondition: startingWord is a single word with no spaces.
   */
  public MessageBuilder(String startingWord)
  { /* to be implemented in part (a) */}

  /**
   *  Returns a word to follow the word specified by the
   *  parameter or null if there are no remaining words.
   *  Precondition: s is a single word with no spaces.
   *  Postcondition: Returns an individual word with no spaces.
   */
  public String getNextWord(String s)
  { /* implementation not shown */ }

  /**
   *  Returns an abbreviation for the instance variable message,
   *  as described in part (b)
   *  Preconditions: Each word in message is separated by a single space.
   *                 message contains two or more words.
   *  Postcondition: message is unchanged.
   */
  public String getAbbreviation()
  { /* to be implemented in part (b) */}

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

**3.** Write the MessageBuilder constructor, which uses a helper method to create a message starting with the word specified by the parameter startingWord and assigns the message to the instance variable message. The constructor also counts the number of words in the message and assigns the count to the instance variable numWords.

Each word in the message will be separated by a single space.

A helper method, getNextWord, has been provided to obtain words to be added to the message. Each call to getNextWord returns the next word in the message based on the word most recently added to the message. When there are no more words to be added to the message, getNextWord returns null. Consecutive calls to getNextWord are guaranteed to eventually return null.

**Example 1**

Consider the following calls to getNextWord, which are made within the MessageBuilder class. The return value of each call is used in the next call.

| Method Call | Return Value |
|---|---|
| getNextWord("the") | "book" |
| getNextWord("book") | "on" |
| getNextWord("on") | "the" |
| getNextWord("the") | "table" |
| getNextWord("table") | null |

Based on these return values, a call to the MessageBuilder constructor with argument "the" should set the instance variable message to "the book on the table" and should set the instance variable numWords to 5.

**Example 2**

Consider the following calls to getNextWord, which are made within the MessageBuilder class. The return value of each call is used in the next call.

## FINAL Unit1-Unit2 FRQ - Topic order

| Method Call | Return Value |
|---|---|
| getNextWord("good") | "morning" |
| getNextWord("morning") | "sunshine" |
| getNextWord("sunshine") | null |

Based on these return values, a call to the MessageBuilder constructor with argument "good" should set the instance variable message to "good morning sunshine" and should set the instance variable numWords to 3.

Complete the MessageBuilder constructor. You must use getNextWord appropriately in order to receive full credit.

```
/**
 * Builds a message starting with the word specified by the parameter and
 * counts the number of words in the message, as described in part (a)
 * Precondition: startingWord is a single word with no spaces.
 */
public MessageBuilder(String startingWord)
```

### Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getNextWord` on the value of `startingWord`

**Canonical Solution:**

```
public MessageBuilder(String startingWord)
{
   message = startingWord;
   String w = getNextWord(startingWord);
   numWords = 1;
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   while(w != null)
   {
      message += " " + w;
      numWords++;
      w = getNextWord(w);
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to `getNextWord`

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loops until no more words remaining

**Canonical Solution:**

```
public MessageBuilder(String startingWord)
{
   message = startingWord;
   String w = getNextWord(startingWord);
   numWords = 1;

   while(w != null)
   {
      message += " " + w;
      numWords++;
      w = getNextWord(w);
   }
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make an incorrect comparison to `null`

### Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Increments a count of the number of words accessed (in the context of a loop)

**Canonical Solution:**

```
public MessageBuilder(String startingWord)
{
   message = startingWord;
   String w = getNextWord(startingWord);
   numWords = 1;

   while(w != null)
   {
      message += " " + w;
      numWords++;
      w = getNextWord(w);
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to assign the count to `numWords`
- fail to initialize `numWords`

## FINAL Unit1-Unit2 FRQ - Topic order

### Part A Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- `message` is correctly built from `startingWord` and the strings obtained from `getNextWord` and `numWords` is assigned the correct count (*algorithm*)

**Canonical Solution:**

```
public MessageBuilder(String startingWord)
{
   message = startingWord;
   String w = getNextWord(startingWord);
   numWords = 1;

   while(w != null)
   {
      message += " " + w;
      numWords++;
      w = getNextWord(w);
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- fail to initialize `message`
- calculate an incorrect count
- include an additional space before the first word or after the last word

## FINAL Unit1-Unit2 FRQ - Topic order

**4.** Write the getAbbreviation method, which returns an abbreviation consisting of the first letter of each word in message. Assume that message consists of two or more words, each separated by a single space.

For example, if the value of message is "as soon as possible", then getAbbreviation() should return "asap".

Complete the getAbbreviation method.

```
/**
 * Returns an abbreviation for the instance variable message,
 * as described in part (b)
 * Preconditions: Each word in message is separated by a single space.
 *                message contains two or more words.
 * Postcondition: message is unchanged.
 */
public String getAbbreviation()
```

---

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loop accesses all necessary parts of `message`

**Canonical Solution:**

```
public String getAbbreviation()
{
   String temp = message;
   String result = temp.substring(0, 1);

   while (temp.indexOf(" ") >= 0)
   {
      int i = temp.indexOf(" ");
      temp = temp.substring(i + 1);
      result += temp.substring(0, 1);
   }
   return result;
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to extract the first letter of the word in a word-based loop

Note: Solutions to part (b) that iterate over the string character-by-character or use the `split` method are valid and can be scored using these scoring guidelines.

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies a letter that appears after a space within message (and all `String` method calls are syntactically valid)

**Canonical Solution**

```
public String getAbbreviation()
{
   String temp = message;
   String result = temp.substring(0, 1);

   while (temp.indexOf(" ") >= 0)
   {
      int i = temp.indexOf(" ");
      temp = temp.substring(i + 1);
      result += temp.substring(0, 1);
   }
   return result;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Note: Solutions to part (b) that iterate over the string character-by-character or use the `split` method are valid and can be scored using these scoring guidelines.

## FINAL Unit1-Unit2 FRQ - Topic order

### Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns string consisting of the first letter of each word in `message` (*algorithm*)

**Canonical Solution:**

```
public String getAbbreviation()
{
   String temp = message;
   String result = temp.substring(0, 1);

   while (temp.indexOf(" ") >= 0)
   {
      int i = temp.indexOf(" ");
      temp = temp.substring(i + 1);
      result += temp.substring(0, 1);
   }
   return result;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- modify `message` or `numWords`

Note: Solutions to part (b) that iterate over the string character-by-character or use the `split` method are valid and can be scored using these scoring guidelines.

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves writing two code segments that use information about downloaded songs.

A record company uses the following variables to maintain information about each song in the company's catalog.

- `downloads` is an `int` variable that represents the number of times a song was downloaded. Assume the value of this variable is positive.
- `songInfo` is a `String` variable that contains information about the song. The string consists of the artist name, followed by a hyphen ("-"), followed by the album title, followed by another hyphen, followed by the song title. Assume that no other hyphens appear in the string.

**5.** A code segment, which you will write, should determine whether the ones digit of the number stored in `downloads` appears exactly one other time in the number. If so, the code segment should print `"exactly once"`; otherwise, it should print `"not exactly once"`.

For example, if `downloads` has a value of `1734567`, then the segment should print `"exactly once"` because the ones digit, which is 7, appears one other time.

As another example, if `downloads` has a value of `325322`, then the segment should print `"not exactly once"` because the ones digit, which is 2, appears two other times.

Write a code segment to print the intended output. Your implementation must conform to the examples.

**Part A Point 1**

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies at least one digit in `downloads`

**Canonical Solution:**

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
}

if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- do not store the value of the ones digit in a variable

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all necessary digits in `downloads`

**Canonical Solution:**

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- fail to initialize counter

Responses will not earn the point if they

- fail to update counter

### Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines if `downloads` has exactly one occurrence of the ones digit

**Canonical Solution:**

```
int onesDigit = downloads % 10;
int remainingDigits = downloads / 10;
int counter = 0;

while (remainingDigits > 0)
{
   int last = remainingDigits % 10;
   remainingDigits = remainingDigits / 10;
   if (onesDigit == last)
   {
      counter++;
   }
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
if (counter == 1)
{
   System.out.println("exactly once");
}
else
{
   System.out.println("not exactly once");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- fail to print the message

## FINAL Unit1-Unit2 FRQ - Topic order

**6.** A code segment, which you will write, should process the string songInfo and print the song title and artist name separated by the string (" by ").

For example, if songInfo is "Carrie Ferriero-Greatest Hits-Tell Me Why", then the code segment should print "Tell Me Why by Carrie Ferriero".

As another example, if songInfo is "Aniita-Mixtape 2-How Much More", then the code segment should print "How Much More by Aniita".

Write a code segment to print the intended output. Your implementation must conform to the examples.

---

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies the artist name from the song information string

**Canonical Solution:**

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- incorrectly identify the location of the first hyphen ("-")

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

## FINAL Unit1-Unit2 FRQ - Topic order

- Identifies the song title from the artist information string

**Canonical Solution:**

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- incorrectly identify the location of the second hyphen ("-")

### Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Correctly prints the identified artist and song title

**Canonical Solution:**

```
int posDash = songInfo.indexOf("-");
String artist = songInfo.substring(0, posDash);
String remaining = songInfo.substring(posDash + 1);
posDash = remaining.indexOf("-");
String song = remaining.substring(posDash + 1);
System.out.println(song + " by " + artist);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

## FINAL Unit1-Unit2 FRQ - Topic order

- incorrectly identify the artist name or song title

Responses will not earn the point if they

- print the artist and song title in an incorrect format

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves processing strings of text.

Consider the following variables.

- colWidth is an int variable with a positive value.
- text is a String variable with a non-null value.
- otherText is a String variable with a non-null value.

## FINAL Unit1-Unit2 FRQ - Topic order

**7.** A code segment, which you will write, should print output according to the following rules.

- If the length of text is greater than colWidth, then the value of text will be printed.
- Otherwise, the printed output will consist of colWidth characters contained between two "|" characters. The text between the "|" characters will consist of a sequence of periods ("."), followed by the value of text, followed by another sequence of periods.

  The printed output will have an equal number of periods on each side of text, if possible. If this is not possible, then there will be one fewer period on the left side of text than on the right side of text.

The intended behavior is illustrated in the following table.

| Value of colWidth | Value of text | Printed Output |
|---|---|---|
| **5** | **"ab"** | \|.ab..\| |
| **5** | **"abc"** | \|.abc.\| |
| **10** | **"ab"** | \|....ab....\| |
| **10** | **"abc"** | \|...abc....\| |
| **10** | **"abcdefghij"** | \|abcdefghij\| |
| **10** | **"abcdefghijklmnopqrstuv"** | abcdefghijklmnopqrstuv |

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

### Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Prints the original text if the text length is longer than the width of the column

## FINAL Unit1-Unit2 FRQ - Topic order

**Canonical Solution:**

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method
- use an incorrect comparison

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies the number of periods needed before and after the text

**Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

| 0 | ✓ 1 |
|---|---|

The response earns the point.

Responses can still earn the point if they

- incorrectly determine the length of the text

### Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Prints string in appropriate format with identified number of periods before and after the text (*algorithm*)

**Canonical Solution:**

```
if (text.length() > colWidth)
{
   System.out.println(text);
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
else
{
   System.out.print("|");
   int padding = (colWidth - text.length()) / 2;
   for (int i = 0; i < padding; i++)
   {
      System.out.print(".");
   }
   System.out.print(text);
   int pos = padding + text.length()
   for (int j = pos; j < colWidth; j++)
   {
      System.out.print(".");
   }
   System.out.println("|");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- print an incorrect number of periods before and after the text

## FINAL Unit1-Unit2 FRQ - Topic order

**8.** A code segment, which you will write, should print the length of the longest prefix of the text that is a substring of otherText. A prefix is a substring that occurs at the beginning of another string.

The intended behavior is illustrated in the following table.

| Value of text | Value of otherText | Printed Output | Explanation |
|---|---|---|---|
| **"apple"** | **"pear"** | 1 | The prefix "a" is a substring of "pear", but "ap" is not. |
| **"apple"** | **"pineapple"** | 5 | The prefixes "a", "ap", "app", "appl", and "apple" are substrings of "pineapple". The length of the longest substring is 5. |
| **"apple"** | **"happy"** | 3 | The prefixes "a", "ap", and "app" are substrings of "happy". The length of the longest substring is 3. |
| **"apple"** | **"leek"** | 0 | No prefix of "apple" is a substring of "leek". |

Write a code segment to print the intended output. Your implementation must meet all specifications and conform to the examples shown in the table.

---

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loop bounds support accessing all necessary parts of `text`

**Canonical Solution:**

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   {
      longest = len;
   }
}
System.out.println(longest);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- use either a condition or an update that would cause an infinite loop or skipped elements in some cases

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Uses an appropriate `String` method to determine whether `text` is a substring of `otherText`

**Canonical Solution:**

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
   {
      longest = len;
   }
}
System.out.println(longest);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

## FINAL Unit1-Unit2 FRQ - Topic order

- make any incorrect call to a `String` method

### Part B Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines the length of the longest prefix (*algorithm*)

**Canonical Solution:**

```
int longest = 0;
for (int len = 1; len <= text.length(); len++)
{
   String prefix = text.substring(0,len);
   if (otherText.indexOf(prefix) > -1)
   {
      longest = len;
   }
}
System.out.println(longest);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- fail to print the result
- loop an incorrect number of times
- make an incorrect call to a `String` method

Responses will not earn the point if they

- fail to update a longest length variable

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves the RobotMover class, which maintains a string that represents the moves made by a robot. You will write one constructor and one method in the RobotMover class.

```
public class RobotMover
{
  private String moveSequence;

  /**
   *  Initalizes moveSequence with numMoves moves, with an equal
   *  chance of "up", "down", "left", and "right". Each move is
   *  followed by an underscore ("_").
   *  Precondition: numMoves > 0
   */
  public RobotMover(int numMoves)
  { /* to be implemented in part (a) */ }

  /**
   *  Returns the number of times that str appears in moveSequence
   *  Precondition:  moveSequence is a valid sequence of moves,
   *                 each followed by an underscore.
   *  Postcondition: moveSequence is unchanged.
   */
  public int countOccurrences(String str)
  { /* to be implemented in part (b) */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

**9.** Write the RobotMover constructor, which initializes the instance variable moveSequence with numMoves moves, with each move followed by an underscore ("_"). Each move should be randomly selected, with an equal chance of being "up", "down", "left", or "right".

For example, new RobotMover(6) will initialize moveSequence with six moves. Some possible values of moveSequence after the call are "up_right_right_left_down_up_" or "up_up_down_down_left_right_".

Complete the RobotMover constructor.

```
/**
 * Initalizes moveSequence with numMoves moves, with an equal
 * chance of "up", "down", "left", and "right". Each move is
 * followed by an underscore ("_").
 * Precondition: numMoves > 0
 */
public RobotMover(int numMoves)
```

### Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calls `Math.random` or an equivalent method to generate a random value

Note: Solutions to part (a) may choose a random direction using a data structure to store the options, as in the following.

```
String[] dirs = {"up_", "down_", "left_", "right_"};
String dir = dirs[(int)(Math.random() * 4)];
```

This is a valid solution strategy and can be scored using these scoring guidelines.

**Canonical Solution:**

```
public RobotMover(int numMoves)
{
   moveSequence = "";
   for (int j = 1; j <= numMoves; j++)
   {
      double prob = Math.random();

      if (prob < 0.25)
      {
         moveSequence += "up_";
      }
      else if (prob < 0.5)
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
      {
         moveSequence += "down_";
      }
      else if (prob < 0.75)
      {
         moveSequence += "left_";
      }
      else
      {
         moveSequence += "right_";
      }
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to save or use the generated value

Responses will not earn the point if they

- fail to call `Math.random` or an equivalent method
- make any incorrect call to `Math.random` or an equivalent method
- call `random` without `Math.`

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loops exactly `numMoves` times (body of control flow statement must not be empty)

Note: Solutions to part (a) may choose a random direction using a data structure to store the options, as in the following.

```
String[] dirs = {"up_", "down_", "left_", "right_"};
String dir = dirs[(int)(Math.random() * 4)];
```

This is a valid solution strategy and can be scored using these scoring guidelines.

**Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
public RobotMover(int numMoves)
{
   moveSequence = "";
   for (int j = 1; j <= numMoves; j++)
   {
      double prob = Math.random();

      if (prob < 0.25)
      {
         moveSequence += "up_";
      }
      else if (prob < 0.5)
      {
         moveSequence += "down_";
      }
      else if (prob < 0.75)
      {
         moveSequence += "left_";
      }
      else
      {
         moveSequence += "right_";
      }
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- return inside the loop, as long as loop bounds would otherwise be correct

### Part A Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies four cases based on comparison(s) with a randomly generated value

Note: Solutions to part (a) may choose a random direction using a data structure to store the options, as in the following.

## FINAL Unit1-Unit2 FRQ - Topic order

```
String[] dirs = {"up_", "down_", "left_", "right_"};
String dir = dirs[(int)(Math.random() * 4)];
```

This is a valid solution strategy and can be scored using these scoring guidelines.

**Canonical Solution:**

```
public RobotMover(int numMoves)
{
   moveSequence = "";
   for (int j = 1; j <= numMoves; j++)
   {
      double prob = Math.random();

      if (prob < 0.25)
      {
         moveSequence += "up_";
      }
      else if (prob < 0.5)
      {
         moveSequence += "down_";
      }
      else if (prob < 0.75)
      {
         moveSequence += "left_";
      }
      else
      {
         moveSequence += "right_";
      }
   }
}
```

| 0 | ✓ 1 |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- use incorrect logic for the condition (e.g. `==` in place of `<` or incorrect constant)
- perform a comparison using incorrectly generated or incorrectly cast random value
- make no explicit comparison, as long as they use a random value to choose among four options

## FINAL Unit1-Unit2 FRQ - Topic order

Responses will not earn the point if they

- use incorrect syntax for the comparison (e.g. `.compareTo` or `=<` in place of `<=`)
- fail to generate a random value

### Part A Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Builds `moveSequence` correctly (*algorithm*)

Note: Solutions to part (a) may choose a random direction using a data structure to store the options, as in the following.

```
String[] dirs = {"up_", "down_", "left_", "right_"};
String dir = dirs[(int)(Math.random() * 4)];
```

This is a valid solution strategy and can be scored using these scoring guidelines.

**Canonical Solution:**

```
public RobotMover(int numMoves)
{
   moveSequence = "";
   for (int j = 1; j <= numMoves; j++)
   {
      double prob = Math.random();

      if (prob < 0.25)
      {
         moveSequence += "up_";
      }
      else if (prob < 0.5)
      {
         moveSequence += "down_";
      }
      else if (prob < 0.75)
      {
         moveSequence += "left_";
      }
      else
      {
         moveSequence += "right_";
      }
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
    }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- compare `double` random values using `<=` or `>=` instead of `<` or `>`

Responses will not earn the point if they

- fail to initialize the instance variable `moveSequence`
- print or return a value instead of or in addition to updating `moveSequence`
- fail to build the string in the correct format, including underscores
- have incorrect probabilities (e.g., due to incorrect casting)

## FINAL Unit1-Unit2 FRQ - Topic order

**10.** Write the countOccurrences method, which returns the number of times that the parameter str appears in moveSequence.

For example, suppose moveSequence is "right_down_left_down_right_down_up_up_up_". For this example, countOccurrences should behave as follows.

- The call countOccurrences("right") should return 2.
- The call countOccurrences("down") should return 3.
- The call countOccurrences("right_d") should return 2.
- The call countOccurrences("up_up") should return 2.

Complete method countOccurrences.

```
/**
 * Returns the number of times that str appears in moveSequence
 * Precondition: moveSequence is a valid sequence of moves,
 *               each followed by an underscore.
 * Postcondition: moveSequence is unchanged.
 */
public int countOccurrences(String str)
```

---

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Loop accesses all necessary positions

**Canonical Solution:**

```
public int countOccurrences(String str)
{
   int count = 0;
   String temp = moveSequence;

   while(temp.indexOf(str) >= 0)
   {
      int loc = temp.indexOf(str);
      temp = temp.substring(loc + 1);
      count++;
   }
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   return count;
}
```

Alternate Canonical:

```
public int countOccurrences(String str)
{
   int occ = 0;
   for (int start = 0; start < moveSequence.length();
        start++)
   {
      if (moveSequence.substring(start).indexOf(str) == 0)
      {
         occ++;
      }
   }
   return occ;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- call any string method incorrectly
- traverse either the string or an array derived from the string

Responses will not earn the point if they

- have an infinite loop in some cases

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies a location of `str` within `moveSequence`

**Canonical Solution:**

```
public int countOccurrences(String str)
{
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   int count = 0;
   String temp = moveSequence;

   while(temp.indexOf(str) >= 0)
   {
      int loc = temp.indexOf(str);
      temp = temp.substring(loc + 1);
      count++;
   }
   return count;
}
```

Alternate Canonical:

```
public int countOccurrences(String str)
{
   int occ = 0;
   for (int start = 0; start < moveSequence.length();
        start++)
   {
      if (moveSequence.substring(start).indexOf(str) == 0)
      {
         occ++;
      }
   }
   return occ;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- use the identified location incorrectly

Responses will not earn the point if they

- make any incorrect call to any string method
- fail to store or use the return value of the string method that identifies the location of the string
- access out-of-bounds characters in any string

### Part B Point 3

## FINAL Unit1-Unit2 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns correct count (*algorithm*)

**Canonical Solution:**

```
public int countOccurrences(String str)
{
   int count = 0;
   String temp = moveSequence;

   while(temp.indexOf(str) >= 0)
   {
      int loc = temp.indexOf(str);
      temp = temp.substring(loc + 1);
      count++;
   }
   return count;
}
```

Alternate Canonical:

```
public int countOccurrences(String str)
{
   int occ = 0;
   for (int start = 0; start < moveSequence.length();
        start++)
   {
      if (moveSequence.substring(start).indexOf(str) == 0)
      {
         occ++;
      }
   }
   return occ;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

## FINAL Unit1-Unit2 FRQ - Topic order

- call any string method incorrectly

Responses will not earn the point if they

- fail to return the result
- modify `moveSequence`

## FINAL Unit1-Unit2 FRQ - Topic order

**11. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.**

Assume that the classes listed in the Java Quick Reference have been imported where appropriate. Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question simulates birds or possibly a bear eating at a bird feeder. The following `Feeder` class contains information about how much food is in the bird feeder and simulates how much food is eaten. You will write two methods of the `Feeder` class.

```
public class Feeder
{
  /**
   * The amount of food, in grams, currently in the bird feeder;
   initialized in the constructor and
   * always greater than or equal to zero
   */
  private int currentFood;
  /**
   * Simulates one day with numBirds birds or possibly a bear at the
   bird feeder,
   * as described in part (a)
   * Precondition: numBirds > 0
   */
  public void simulateOneDay(int numBirds)
  { /* to be implemented in part (a) */ }
  /**
   * Returns the number of days birds or a bear found food to eat at
   the feeder in this simulation,
   * as described in part (b)
   * Preconditions: numBirds > 0, numDays > 0
   */
  public int simulateManyDays(int numBirds, int numDays)
  { /* to be implemented in part (b) */ }
  // There may be instance variables, constructors, or methods that
  are not shown.
 }
```

(a) Write the `simulateOneDay` method, which simulates `numBirds` birds or possibly a bear at the feeder for one day. The method determines the amount of food taken from the feeder on this day and updates the `currentFood` instance variable. The simulation accounts for normal conditions, which occur 95% of the time, and abnormal conditions, which occur 5% of the time.

Under normal conditions, the simulation assumes that on any given day, only birds visit the feeder and that each bird at the feeder consumes the same amount of food. This standard amount consumed is between 10 and 50 grams of food, inclusive, in 1-gram increments. That is, on any given day, each bird

## FINAL Unit1-Unit2 FRQ - Topic order

might eat 10, 11, . . . , 49, or 50 grams of food. The amount of food eaten by each bird on a given day is randomly generated and each integer from 10 to 50, inclusive, has an equal chance of being chosen.

For example, a run of the simulation might predict that for a certain day under normal conditions, each bird coming to the feeder will eat 11 grams of food. If 10 birds come to the feeder on that day, then a total of 110 grams of food will be consumed.

If the simulated food consumed is greater than the amount of food in the feeder, the birds empty the feeder and the amount of food in the feeder at the end of the day is zero.

Under abnormal conditions, a bear empties the feeder and the amount of food in the feeder at the end of the day is zero.

The following examples show possible results of three calls to `simulateOneDay`.

Example 1: If the feeder initially contains 500 grams of food, the call `simulateOneDay(12)` could result in 12 birds eating 20 grams of food each, leaving 260 grams of food in the feeder.

Example 2: If the feeder initially contains 1,000 grams of food, the call `simulateOneDay(22)` could result in a bear eating all the food, leaving 0 grams of food in the feeder.

Example 3: If the feeder initially contains 100 grams of food, the call `simulateOneDay(5)` could result in 5 birds attempting to eat 30 grams of food each. Since the feeder initially contains less than 150 grams of food, the feeder is emptied, leaving 0 grams of food in the feeder.

Complete the `simulateOneDay` method.

```
/**
* Simulates one day with numBirds birds or possibly a bear at the
bird feeder,
* as described in part (a)
* Precondition: numBirds > 0
*/
public void simulateOneDay(int numBirds)
```

Class information for this question

```
public class Feeder

private int currentFood

public void simulateOneDay(int numBirds)
public int simulateManyDays(int numBirds, int numDays)
```

## FINAL Unit1-Unit2 FRQ - Topic order

(b) Write the `simulateManyDays` method. The method uses `simulateOneDay` to simulate `numBirds` birds or a bear coming to the feeder on at most `numDays` consecutive days. The simulation returns the number of days that birds or a bear found food at the feeder.

Consider the following examples.

| Value of `currentFood` and Method Call | Possible Outcomes and Resulting Return Value |
|---|---|
| `currentFood: 2400`<br>`simulateManyDays(10, 4)` | Day 1: `simulateOneDay` leaves 2100 grams of food in the feeder.<br>Day 2: `simulateOneDay` leaves 1650 grams of food in the feeder.<br>Day 3: `simulateOneDay` leaves 1500 grams of food in the feeder.<br>Day 4: `simulateOneDay` leaves 1260 grams of food in the feeder.<br>The simulation returns `4` because, on all four days of the simulation, birds or a bear found food at the feeder. The instance variable `currentFood` has the value `1260`. |
| `currentFood: 250`<br>`simulateManyDays(10, 5)` | Day 1: `simulateOneDay` leaves 150 grams of food in the feeder.<br>Day 2: `simulateOneDay` leaves 0 grams of food in the feeder.<br>The simulation returns `2` because, on two of the five simulated days, birds or a bear found food at the feeder. The instance variable `currentFood` has the value `0`. |
| `currentFood: 0`<br>`simulateManyDays(5, 10)` | The simulation returns `0` because no food was found at the feeder on any day. The instance variable `currentFood` has the value `0`. |

Complete the `simulateManyDays` method. Assume that `simulateOneDay` works as intended, regardless of what you wrote in part (a). You must use `simulateOneDay` appropriately in order to receive full credit.

```
/**
* Returns the number of days birds or a bear found food to eat at
the feeder in this simulation,
* as described in part (b)
* Preconditions: numBirds > 0, numDays > 0
*/
public int simulateManyDays(int numBirds, int numDays)
```

Class information for this question

```
public class Feeder

private int currentFood
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
public void simulateOneDay(int numBirds)
public int simulateManyDays(int numBirds, int numDays)
```

**Part A – simulateOneDay (4 points)**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. $+1$ indicates a point earned.

$+1$ **[Skill 3.A]** Generates a random value

Responses can still earn the point even if they

- fail to save or use the generated random value

Responses will not earn the point if they

- fail to call $\text{Math.random}$, or possibly the equivalent, at least one time
- make any incorrect call to $\text{Math.random}$

$+1$ **[Skill 3.C]** Identifies two cases based on a comparison of a randomly generated value and some constant that implements a 5% probability

Responses can still earn the point even if they

- reverse the $5/95$ probability
- compare $\text{double}$ values using $<=$ or $>=$ instead of $<$ or $>$
- incorrectly cast the $5/95$ random value, as long as a suitable range is generated and the comparison divides that range appropriately
- call $\text{Math.random}$ incorrectly

$+1$ **[Skill 3.C]** Generates a random integer that is uniform in the range $[10, 50]$

Responses can still earn the point even if they

- call $\text{Math.random}$ incorrectly

$+1$ **[Skill 3.C]** Scales for $\text{numBirds}$ and subtracts appropriate amount from $\text{currentFood}$ in all cases (*algorithm*)

Responses can still earn the point even if they

- compare $\text{double}$ values using $<=$ or $>=$ instead of $<$ or $>$

Responses will not earn the point if they

- reverse the $5/95$ probability

## FINAL Unit1-Unit2 FRQ - Topic order

- incorrectly process the $5/95$ range/comparison (e.g by incorrect casting)
- exit the method while currentFood has a negative value

**Canonical Solution:**

```
public void simulateOneDay(int numBirds)
{
   double condition = Math.random();
   if (condition < 0.05)
   {
      currentFood = 0;
   }
   else
   {
      int eachBirdEats =
         (int) (Math.random() * 41) + 10;
      int totalEaten = numBirds * eachBirdEats;
      if (totalEaten > currentFood)
      {
         currentFood = 0;
      }
      else
      {
         currentFood -= totalEaten;
      }
   }
}
```

| 0 | 1 | 2 | 3 | 4 ✓ |
|---|---|---|---|---|

Total number of points earned is equal to 4.

- ☐ $+1$ Generates a random value **(Points Earned)**
- ☐ $+1$ Identifies two cases based on a comparison of a randomly generated value and some constant that implements a 5% probability **(Points Earned)**
- ☐ $+1$ Generates a random integer that is uniform in the range $[10, 50]$ **(Points Earned)**
- ☐ $+1$ Scales for numBirds and subtracts appropriate amount from currentFood in all cases (*algorithm*) **(Points Earned)**

**Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
public void simulateOneDay(int numBirds)
{
   double condition = Math.random();
   if (condition < 0.05)
   {
      currentFood = 0;
   }
   else
   {
      int eachBirdEats =
         (int) (Math.random() * 41) + 10;
      int totalEaten = numBirds * eachBirdEats;
      if (totalEaten > currentFood)
      {
         currentFood = 0;
      }
      else
      {
         currentFood -= totalEaten;
      }
   }
}
```

**Part B – simulateManyDays (5 points)**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned.

+1 **[Skill 3.A]** Calls simulateOneDay with value of numBirds

Responses will not earn the point if they

- fail to make at least one correct call to simulateOneDay
- make any call to simulateOneDay on the class or on an object other than this (use of this is optional)

+1 **[Skill 3.C]** Loops over the simulation method call and guards that it runs at most numDays times

Responses can still earn the point even if they

- call the method that simulates one day incorrectly
- count the simulated days incorrectly, as long as the loop guard would work if the count were corrected
- fail to guard against calls to the simulation method when currentFood <= 0

Responses will not earn the point if they

- fail to count the simulated days at all

## FINAL Unit1-Unit2 FRQ - Topic order

+1 **[Skill 3.C]** Counts the number of times that the method that simulates one day is called with food available in the feeder (*algorithm*)

Responses can still earn the point even if they

- simulate extra days when currentFood is 0, as long as the count is still correct

Responses will not earn the point if they

- fail to initialize the counter appropriately
- compute the correct count but return something else
- fail to test if food is available

+1 **[Skill 3.C]** Compares currentFood and 0

Responses can still earn the point even if they

- fail to make the comparison in the context of a loop
- use the result of the comparison incorrectly

+1 **[Skill 3.B]** Returns any int value, in all cases

Responses can still earn the point even if they

- return a constant

**Canonical Solution:**

```
public int simulateManyDays(int numBirds,
                            int numDays)
{
   for (int daysSoFar = 0; daysSoFar < numDays;
        daysSoFar++)
   {
      if (currentFood == 0)
      {
         return daysSoFar;
      }
      simulateOneDay(numBirds);
   }
   return numDays;
}
```

| 0 | 1 | 2 | 3 | 4 | 5 ✓ |
|---|---|---|---|---|---|

## FINAL Unit1-Unit2 FRQ - Topic order

Total number of points earned is equal to 5.

- ☐ +1 Calls simulateOneDay with value of numBirds **(Points Earned)**
- ☐ +1 Loops over the simulation method call and guards that it runs at most numDays times **(Points Earned)**
- ☐ +1 Counts the number of times that the method that simulates one day is called with food available in the feeder (*algorithm*) **(Points Earned)**
- ☐ +1 Compares currentFood and 0 **(Points Earned)**
- ☐ +1 Returns any int value, in all cases **(Points Earned)**

**Canonical Solution:**

```
public int simulateManyDays(int numBirds,
                            int numDays)
{
   for (int daysSoFar = 0; daysSoFar < numDays;
        daysSoFar++)
   {
      if (currentFood == 0)
      {
         return daysSoFar;
      }
      simulateOneDay(numBirds);
   }
   return numDays;
}
```

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves processing a String that represents a sequence of beads.

The String variable beadSequence represents a bead sequence. It contains a sequence of letters in which each letter represents the color of a bead.

A bead sequence is considered acceptable if it adheres to the following rules. Otherwise, it is not considered acceptable.

- Rule 1: The number of beads in the sequence is between 5 and 10, inclusive.
- Rule 2: The sequence contains at least one blue bead (represented by "B") and at least one white bead (represented by "W").
- Rule 3: The sequence starts and ends with the same color.

You will create three boolean variables to help determine whether the sequence is acceptable, and then print a message indicating the result. The following table illustrates the intended behavior for different values of beadSequence.

| Value of beadSequence | Meets Rule 1 | Meets Rule 2 | Meets Rule 3 | Printed Output |
|---|---|---|---|---|
| **"OBWO"** | false | true | true | "Not acceptable" |
| **"BWBGOGWBB"** | true | true | true | "Acceptable" |
| **"ROYGBIVW"** | true | true | false | "Not acceptable" |
| **"BOBOBOBO"** | true | false | false | "Not acceptable" |
| **"WBWRWBWRWBWR"** | false | true | false | "Not acceptable" |
| **"BWWWBBWWWB"** | true | true | true | "Acceptable" |

## FINAL Unit1-Unit2 FRQ - Topic order

**12.** Write a code segment that creates a boolean variable named correctColors and assigns it true if beadSequence meets rule 2 and assigns it false otherwise.

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Uses an appropriate `String` method to determine the bead sequence contains at least one `"B"` and at least one `"W"`

**Canonical Solution:**

```
boolean correctColors;
if ((beadSequence.indexOf("B") == -1) ||
    (beadSequence.indexOf("W") == -1))
{
   correctColors = false;
}
else
{
   correctColors = true;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method
- fail to test for both `"B"` and `"W"`

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns the appropriate `boolean` value to `correctColors` based on the values `"B"` and `"W"` both appearing in the bead sequence

**Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
boolean correctColors;
if ((beadSequence.indexOf("B") == -1) ||
    (beadSequence.indexOf("W") == -1))
{
   correctColors = false;
}
else
{
   correctColors = true;
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- make any incorrect call to a `String` method
- fail to test for both `"B"` and `"W"`

Responses will not earn the point if they

- fail to check for letters appearing in the bead sequence

## FINAL Unit1-Unit2 FRQ - Topic order

**13.** Write a code segment that creates a boolean variable named lengthGood and assigns it true if beadSequence meets rule 1 and assigns it false otherwise.

### Part A Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Finds the length of the bead sequence using appropriate `String` method call

**Canonical Solution:**

```
int len = beadSequence.length();
boolean lengthGood = (len >= 5 && len <= 10);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method

### Part A Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns the appropriate `boolean` value to `lengthGood` based on the length of the bead sequence

**Canonical Solution:**

```
int len = beadSequence.length();
boolean lengthGood = (len >= 5 && len <= 10);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

## FINAL Unit1-Unit2 FRQ - Topic order

- incorrectly determine the length of the bead sequence

Responses will not earn the point if they

- fail to compare against the length of `5` and the length of `10`

## FINAL Unit1-Unit2 FRQ - Topic order

**14.** Write a code segment that creates a boolean variable named startEndGood and assigns it true if beadSequence meets rule 3 and assigns it false otherwise.

### Part C Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Uses an appropriate `String` method call to identify the first and last letters of the bead sequence

**Canonical Solution:**

```
String first = beadSequence.substring(0, 1);
String last = beadSequence.substring(len - 1);
boolean startEndGood = (first == last);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method
- fail to consider both first and last letters

### Part C Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns the appropriate `boolean` value to `startEndGood` based on the identified first and last letters

**Canonical Solution:**

```
String first = beadSequence.substring(0, 1);
String last = beadSequence.substring(len - 1);
boolean startEndGood = (first == last);
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

## FINAL Unit1-Unit2 FRQ - Topic order

Responses can still earn the point if they

- make an incorrect call to a `String` method
- incorrectly identify first or last letter

Responses will not earn the point if they

- fail to consider both first and last letters

## FINAL Unit1-Unit2 FRQ - Topic order

**15.** Write a code segment that prints "Acceptable" if beadSequence adheres to all three rules and prints "Not acceptable" otherwise. You must use the variables lengthGood, correctColors, and startEndGood to receive full credit, regardless of what you wrote for parts A, B, and C.

---

### Part D Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines if bead sequence is acceptable using the variables `lengthGood`, `correctColors`, and `startEndGood`

**Canonical Solution:**

```
if (lengthGood && correctColors && startEndGood)
{
   System.out.println("Acceptable");
}
else
{
   System.out.println("Not acceptable");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- fail to use `lengthGood`, `correctColors`, and `startEndGood` variables

### Part D Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Prints message based on determination of whether bead sequence is acceptable

**Canonical Solution:**

```
if (lengthGood && correctColors && startEndGood)
{
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
   System.out.println("Acceptable");
}
else
{
   System.out.println("Not acceptable");
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point if they

- incorrectly determine if the bead sequence is acceptable

Responses will not earn the point if they

- fail to print the message

## FINAL Unit1-Unit2 FRQ - Topic order

**16. Directions: SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.**

Notes:

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

4. This question involves analyzing strings. You will write the `countRepeat` method of the following `StringAnalyzer` class.

```
public class StringAnalyzer
{
 /** Returns a count of how many times smallStr occurs in largeStr,
 as
  *   described in part (a)
  *   Precondition: largeStr is not null; smallStr is not null.
  *                 The length of smallStr is less than the length of
 largeStr.
  */
 public static int countRepeat(String largeStr, String smallStr)
 {  /* to be implemented in part (a) */  }

 // There may be variables and methods that are not shown.
}
```

(a) The `countRepeat` method is used to count and return the number of times the string parameter `smallStr` appears in the string parameter `largeStr`. In the case of overlapping occurrences of `smallStr`, only the first occurrence is included in the overall count, as shown in the second and third calls to `countRepeat` in the following table of examples.

| **Call to** `countRepeat` | **Return Value** |
|---|---|
| `StringAnalyzer.countRepeat("BAAB", "AA")` | 1 |
| `StringAnalyzer.countRepeat("AAAAA", "AA")` | 2 |
| `StringAnalyzer.countRepeat("AABABABAA", "ABA")` | 2 |
| `StringAnalyzer.countRepeat("ABBAABB", "ABA")` | 0 |

Complete method `countRepeat`.

```
/** Returns a count of how many times smallStr occurs in largeStr,
 *   as described in part (a)
 *   Precondition: largeStr is not null; smallStr is not null.
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
     *               The length of smallStr is less than the length of
largeStr.
     */
public static int countRepeat(String largeStr, String smallStr)
```

(b) A programmer would like to modify the `StringAnalyzer` class to limit the length of the large string that can be passed as input to `countRepeat`. The limit could vary between calls to `countRepeat`. A call to `countRepeat` with a value of `largeStr` that is longer than the maximum length would return `-1`. The programmer would like to implement this change without making any changes to the signature of the `countRepeat` method or overloading `countRepeat`.

Write a description of how you would change the `StringAnalyzer` class in order to support this modification. **Do not write the program code for this change.**

Make sure to include the following in your response.

- Identify any new or modified variables or methods.
- Describe, for each new or revised variable or method, how it would change or be implemented, including visibility and type.

**Part A – countRepeat**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned and -1 indicates a general or question-specific penalty.

**+1 [Skill 3.C]** Traverses largeStr (*no bounds errors*)

Responses will not earn the point if they:

· do not use largeStr in the loop

· calculate the length of smallStr incorrectly for use in the traversal of largeStr

**+1 [Skill 3.A]** Gets a substring of length of smallStr from largeStr

Responses can still earn the point even if they calculate the length of smallStr using an incorrect call to length

Responses will not earn the point if they do not use the length of smallStr to compare substrings of equal length

**+1 [Skill 3.A]** Uses smallStr and a substring of largeStr to guard the increment of the counter and update of the loop control variable

Responses can still earn the point even if they:

· get an incorrect substring from largeStr

· use incorrect logic to guard the increments

Responses will not earn the point if they use == instead of .equals

**+1 [Skill 3.C]** Updates loop control variable and counter based on result of comparison (algorithm)

## FINAL Unit1-Unit2 FRQ - Topic order

Responses can still earn the point even if they calculate the length of smallStr using an incorrect call to length

Responses will not earn the point if they:

· do not initialize the loop control variable or counter

· use incorrect logic to guard the increments

· update loop control variable by something other than the length of smallStr

· count overlapping occurrences (for a reason other than incorrect smallStr length)

**+1 [Skill 3.C]** Returns count of non-overlapping substring occurrences

Responses can still earn the point even if they:

· accumulate count incorrectly

· have a bounds error

Responses will not earn the point if they return early

**-1** (w) Extraneous code that causes side-effect (e.g., printing to output, incorrect precondition check)

**-1** (x) Local variables used but none declared

**Canonical Solution:**

```
public static int countRepeat(String largeStr,
                              String smallStr)
{
   int count = 0;
   int i = 0;
   int len = smallStr.length();

   while (i < largeStr.length() - len + 1)
   {
      if (smallStr.equals(largeStr.substring(i,
         i + len)))
      {
         count++;
         i += len;
      }
      else
      {
         i++;
      }
   }
   return count;
}
```

**Alternate Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
public static int countRepeat(String largeStr,
                              String smallStr)
{
   int count = 0;
   String word = largeStr;
   int len = smallStr.length();
   int ind = word.indexOf(smallStr);

   while (ind != -1)
   {
      count++;
      word = word.substring(ind + len);
      ind = word.indexOf(smallStr);
   }

   return count;
}
```

| 0 | 1 | 2 | 3 | 4 | 5 ✓ |
|---|---|---|---|---|---|

Total number of points earned (minus penalties) is equal to 5.

- ☐ **+1** Traverses largeStr (*no bounds errors*) **(Points earned)**
- ☐ **+1** Gets a substring of length of smallStr from largeStr **(Points earned)**
- ☐ **+1** Uses smallStr and a substring of largeStr to guard the increment of the counter and update of the loop control variable **(Points earned)**
- ☐ **+1** Updates loop control variable and counter based on result of comparison (algorithm) **(Points earned)**
- ☐ **+1** Returns count of non-overlapping substring occurrences **(Points earned)**
- ☐ **-1** [penalty] (w) Extraneous code that causes side-effect (e.g., printing to output, incorrect precondition check) **(General penalties)**
- ☐ **-1** [penalty] (x) Local variables used but none declared **(General penalties)**

**Canonical Solution:**

## FINAL Unit1-Unit2 FRQ - Topic order

```
public static int countRepeat(String largeStr,
                              String smallStr)
{
   int count = 0;
   int i = 0;
   int len = smallStr.length();

   while (i < largeStr.length() - len + 1)
   {
      if (smallStr.equals(largeStr.substring(i,
         i + len)))
      {
         count++;
         i += len;
      }
      else
      {
         i++;
      }
   }
   return count;
}
```

**Alternate Canonical Solution:**

```
public static int countRepeat(String largeStr,
                              String smallStr)
{
   int count = 0;
   String word = largeStr;
   int len = smallStr.length();
   int ind = word.indexOf(smallStr);

   while (ind != -1)
   {
      count++;
      word = word.substring(ind + len);
      ind = word.indexOf(smallStr);
   }

   return count;
}
```

**Part B – StringAnalyzer**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned and -1 indicates a general or question-specific penalty.

**+1 [Skill 3.B]** The response identifies properties of the new variable:

· int or "integer"

· not local to a method

## FINAL Unit1-Unit2 FRQ - Topic order

Responses can still earn the point even if they:

o specify the variable as an instance variable

o do not describe a variable or identify a type, but describe an appropriate identified value as generated only once during the method call, randomly or from user input

Responses will not earn the point if they describe a change to the method header

**+1 [Skill 3.B]** The response describes how the method changes:

· method `countRepeat` would need to change so that it checks if the length of `largeStr` is less than the static variable and returns $-1$ if so

Responses can still earn the point even if they describe a change to the method header

Responses only earn the point if all described components are plausible, accurate, and consistent as related to the problem

| 0 | 1 | ✓ 2 |
|---|---|---|

Total number of points earned (minus penalties) is equal to 2.

☐ **+1** The response identifies properties of the new variable **(Points earned)**:
o `int` or "integer"
o not local to a method

☐ **+1** The response describes how the method changes **(Points earned)**:
o method `countRepeat` would need to change so that it checks if the length of `largeStr` is less than the static variable and returns $-1$ if so

## FINAL Unit1-Unit2 FRQ - Topic order

This question involves processing an email address to create three usernames.

The email address is stored in a String variable named email, which consists of a first name, followed by ".", followed by a last name, followed by "@", followed by a domain name. Assume that the email address is stored in this format and the last name has at least three characters.

The usernames will be formatted as follows.

- Username 1 will consist of the first name, followed by "_", followed by the last name.
- Username 2 will consist of the first initial of the first name, followed by "_", followed by the last name.
- Username 3 will consist of a randomly-selected character from the first name , followed by "_", followed by a sequence of three consecutive characters selected from a random position in the last name.

Example 1

Suppose email is equal to "joseph.martin@school.org".

- Username 1 will be "joseph_martin".
- Username 2 will be "j_martin".
- Possible values of Username 3 include "s_art" and "p_tin".

Example 2

Suppose email is equal to "virginia.farmingham@school.net".

- Username 1 will be "virginia_farmingham".
- Username 2 will be "v_farmingham".
- Possible values of Username 3 include "g_ham" and "i_arm".

**17.** Write a statement that uses the String method indexOf to find the index of the first "." in email and assign it to a variable. Write another statement that uses the String method indexOf to find the index of "@" and assign it to a variable.

**Part A Point 1**

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Finds the positions of the first period `"."` and the `"@"` using appropriate `String` method calls

## FINAL Unit1-Unit2 FRQ - Topic order

**Canonical Solution:**

```
int periodLoc = email.indexOf(".");
int atLoc = email.indexOf("@");
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method

## FINAL Unit1-Unit2 FRQ - Topic order

**18.** Write a statement that uses the String method substring to extract the first name from email and assign it to a variable. Write another statement that uses the String method substring to extract the last name from email and assign it to a variable. Then use the results of those statements to create and assign the correct value to a String variable for username 1.

### Part B Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Extracts the first and last names from the email address

**Canonical Solution:**

```
String first = email.substring(0, periodLoc);
String last = email.substring(periodLoc + 1, atLoc);
String user1 = first + "_" + last;
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- incorrectly identify the location of the first period `"."` or the `"@"`

Responses will not earn the point if they

- make any incorrect call to a `String` method

### Part B Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Creates a string variable that contains first name and last name

**Canonical Solution:**

```
String first = email.substring(0, periodLoc);
String last = email.substring(periodLoc + 1, atLoc);
String user1 = first + "_" + last;
```

## FINAL Unit1-Unit2 FRQ - Topic order

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- omit the underscore (`"_"`)

Responses will not earn the point if they

- make any incorrect call to a `String` method

## FINAL Unit1-Unit2 FRQ - Topic order

**19.** Write a statement that uses the String method substring to extract the first initial of the first name from email and assign it to a variable. Use the results of this statement and the statements you wrote in part B to create and assign the correct value to a String variable for username 2.

### Part C Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Extracts the first initial of the first name

**Canonical Solution:**

```
String firstInit = email.substring(0, 1);
String user2 = firstInit + "_" + last;
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- make any incorrect call to a `String` method

### Part C Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Creates a string variable that contains the first initial of the first name and the last name

**Canonical Solution:**

```
String firstInit = email.substring(0, 1);
String user2 = firstInit + "_" + last;
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

## FINAL Unit1-Unit2 FRQ - Topic order

- omit the underscore (`"_"`)

Responses will not earn the point if they

- make any incorrect call to a `String` method

## FINAL Unit1-Unit2 FRQ - Topic order

**20.** Write a code segment that uses the random numbers that you generated in part D to create and assign the correct value to a String variable for username 3.

---

### Part E Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Extracts a randomly selected character from the first name and randomly selected sequence of 3 characters from the last name

**Canonical Solution:**

```
String firstChar = first.substring(numFirst, numFirst + 1);
String lastChar3 = last.substring(numLast, numLast + 3);
String user3 = firstChar + "_" + lastChar3;
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- incorrectly generate a random number

Responses will not earn the point if they

- make any incorrect call to a `String` method

### Part E Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Creates a string variable containing identified character from the first name and identified characters from the last name

**Canonical Solution:**

```
String firstChar = first.substring(numFirst, numFirst + 1);
String lastChar3 = last.substring(numLast, numLast + 3);
String user3 = firstChar + "_" + lastChar3;
```

## FINAL Unit1-Unit2 FRQ - Topic order

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- omit the underscore (`"_"`)
- incorrectly obtain the character from the first name or the three characters from the last name

Responses will not earn the point if they

- make any incorrect call to a `String` method

## FINAL Unit1-Unit2 FRQ - Topic order

**21.** SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the `AppointmentBook` class, which provides methods for students to schedule appointments with their teacher. Appointments can be scheduled during one of eight class periods during the school day, numbered 1 through 8. A requested appointment has a duration, which is the number of minutes the appointment will last. The 60 minutes within a period are numbered 0 through 59. In order for an appointment to be scheduled, the teacher must have a block of consecutive, available minutes that contains at least the requested number of minutes in a requested period. Scheduled appointments must start and end within the same period.

The `AppointmentBook` class contains two helper methods, `isMinuteFree` and `reserveBlock`. You will write two additional methods of the `AppointmentBook` class.

```
public class AppointmentBook
{
 /**
  * Returns true if minute in period is available for an appointment
  and returns
  * false otherwise
  * Preconditions: 1 <= period <= 8; 0 <= minute <= 59
  */
 private boolean isMinuteFree(int period, int minute)
 { /* implementation not shown */ }


 /**
  * Marks the block of minutes that starts at startMinute in period
  and
  * is duration minutes long as reserved for an appointment
  * Preconditions: 1 <= period <= 8; 0 <= startMinute <= 59;
  * 1 <= duration <= 60
  */
 private void reserveBlock(int period, int startMinute, int
 duration)
 { /* implementation not shown */ }


 /**
  * Searches for the first block of duration free minutes during
  period, as described in
  * part (a). Returns the first minute in the block if such a block
  is found or returns -1 if no
```

## FINAL Unit1-Unit2 FRQ - Topic order

```
     * such block is found.
     * Preconditions: 1 <= period <= 8; 1 <= duration <= 60
     */
    public int findFreeBlock(int period, int duration)
    { /* to be implemented in part (a) */ }


    /**
     * Searches periods from startPeriod to endPeriod, inclusive, for a
     block
     * of duration free minutes, as described in part (b). If such a
     block is found,
     * calls reserveBlock to reserve the block of minutes and returns
     true; otherwise
     * returns false.
     * Preconditions: 1 <= startPeriod <= endPeriod <= 8; 1 <= duration
     <= 60
     */
    public boolean makeAppointment(int startPeriod, int endPeriod,
                                                    int duration)
    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, and methods that
    are not shown.
}
```

Write the `findFreeBlock` method, which searches `period` for the first block of free minutes that is `duration` minutes long. If such a block is found, `findFreeBlock` returns the first minute in the block. Otherwise, `findFreeBlock` returns `-1`. The `findFreeBlock` method uses the helper method `isMinuteFree`, which returns `true` if a particular minute is available to be included in a new appointment and returns `false` if the minute is unavailable.

Consider the following list of unavailable and available minutes in period 2.

| Minutes in Period 2 | Available? |
|---|---|
| 0 – 9 (10 minutes) | No |
| 10 – 14 (5 minutes) | Yes |
| 15 – 29 (15 minutes) | No |
| 30 – 44 (15 minutes) | Yes |
| 45 – 49 (5 minutes) | No |
| 50 – 59 (10 minutes) | Yes |

The method call `findFreeBlock(2, 15)` would return `30` to indicate that a 15-minute block starting with minute `30` is available. No steps should be taken as a result of the call to `findFreeBlock` to mark those 15 minutes as unavailable.

The method call `findFreeBlock(2, 9)` would also return `30`. Whenever there are multiple blocks that satisfy the requirement, the earliest starting minute is returned.

## FINAL Unit1-Unit2 FRQ - Topic order

The method call `findFreeBlock(2, 20)` would return `-1`, since no 20-minute block of available minutes exists in period 2.

Complete method `findFreeBlock`. You must use `isMinuteFree` appropriately in order to receive full credit.

```
/**
* Searches for the first block of duration free minutes during
period, as described in
* part (a). Returns the first minute in the block if such a block is
found or returns -1 if no
* such block is found.
* Preconditions: 1 <= period <= 8; 1 <= duration <= 60
*/
public int findFreeBlock(int period, int duration)
```

## FINAL Unit1-Unit2 FRQ - Topic order

Write the `makeAppointment` method, which searches the periods from `startPeriod` to `endPeriod`, inclusive, for the earliest block of `duration` available minutes in the lowest-numbered period. If such a block is found, the `makeAppointment` method calls the helper method `reserveBlock` to mark the minutes in the block as unavailable and returns `true`. If no such block is found, the `makeAppointment` method returns `false`.

Consider the following list of unavailable and available minutes in periods 2, 3, and 4 and three successive calls to `makeAppointment`.

| Period | Minutes | Available? |
|---|---|---|
| 2 | 0 – 24 (25 minutes) | No |
| 2 | 25 – 29 (5 minutes) | Yes |
| 2 | 30 – 59 (30 minutes) | No |
| 3 | 0 – 14 (15 minutes) | Yes |
| 3 | 15 – 40 (26 minutes) | No |
| 3 | 41 – 59 (19 minutes) | Yes |
| 4 | 0 – 4 (5 minutes) | No |
| 4 | 5 – 29 (25 minutes) | Yes |
| 4 | 30 – 43 (14 minutes) | No |
| 4 | 44 – 59 (16 minutes) | Yes |

The method call `makeAppointment(2, 4, 22)` returns `true` and results in the minutes 5 through 26, inclusive, in period 4 being marked as unavailable.

The method call `makeAppointment(3, 4, 3)` returns `true` and results in the minutes 0 through 2, inclusive, in period 3 being marked as unavailable.

The method call `makeAppointment(2, 4, 30)` returns `false`, since there is no block of 30 available minutes in periods 2, 3, or 4.

The following shows the updated list of unavailable and available minutes in periods 2, 3, and 4 after the three example method calls are complete.

| Period | Minutes | Available? |
|---|---|---|
| 2 | 0 – 24 (25 minutes) | No |
| 2 | 25 – 29 (5 minutes) | Yes |
| 2 | 30 – 59 (30 minutes) | No |
| 3 | 0 – 2 (3 minutes) | No |
| 3 | 3 – 14 (12 minutes) | Yes |
| 3 | 15 – 40 (26 minutes) | No |
| 3 | 41 – 59 (19 minutes) | Yes |
| 4 | 0 – 26 (27 minutes) | No |
| 4 | 27 – 29 (3 minutes) | Yes |

## FINAL Unit1-Unit2 FRQ - Topic order

| Period | Minutes | Available? |
|---|---|---|
| 4 | 30 – 43 (14 minutes) | No |
| 4 | 44 – 59 (16 minutes) | Yes |

Complete method `makeAppointment`. Assume that `findFreeBlock` works as intended, regardless of what you wrote in part (a). You must use `findFreeBlock` and `reserveBlock` appropriately in order to receive full credit.

```
/**
* Searches periods from startPeriod to endPeriod, inclusive, for a
block
* of duration free minutes, as described in part (b). If such a
block is found,
* calls reserveBlock to reserve the block of minutes and returns
true; otherwise
* returns false.
* Preconditions: 1 <= startPeriod <= endPeriod <= 8; 1 <= duration
<= 60
*/
public boolean makeAppointment(int startPeriod, int endPeriod,
                                                int duration)
```

**Part A – findFreeBlock (5 points)**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned and −1 indicates a general or question-specific penalty.

+1 **[Skill 3.C]** Loops over necessary minutes in an hour

Responses can still earn the point even if they

- loop over fewer than 60 minutes as long as at least (60 – duration + 1) minutes are included
- loop over 60 minutes and use a boolean to indicate that a free block has been found

+1 **[Skill 3.A]** Calls isMinuteFree with period and another int parameter

Responses can still earn the point even if they

- call isMinuteFree with invalid parameters due to incorrect loop bounds

Responses will not earn the point if they

- use incorrect parameter types
- order the parameters incorrectly
- call the method on the class or on an object other than this (use of this is optional)

## FINAL Unit1-Unit2 FRQ - Topic order

+1 **[Skill 3.C]** Keeps track of contiguous free minutes in a block (*algorithm*)

Responses can still earn the point even if they

- call isMinuteFree incorrectly

Responses will not earn the point if they

- fail to reset when a nonfree minute is found
- call isMinuteFree with minutes >= 60

+1 **[Skill 3.C]** Checks whether a valid block of duration minutes has been found

Responses can still earn the point even if they

- maintain a boolean instead of accumulating the block length

+1 **[Skill 3.C]** Calculates and returns starting minute and −1 appropriately based on identified block (*algorithm*)

Responses will not earn the point if they

- are off by one on the returned value

**Canonical Solution:**

```
public int findFreeBlock(int period, int duration)
{
   int blockLength = 0;

   for (int minute = 0; minute < 60; minute++)
   {
      if (isMinuteFree(period, minute))
      {
         blockLength++;
         if (blockLength == duration)
         {
            return minute - blockLength + 1;
         }
      }
      else
      {
         blockLength = 0;
      }
   }
   return -1;
}
```



| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|

## FINAL Unit1-Unit2 FRQ - Topic order

Total number of points earned (minus penalties) is equal to 5.

- ☐ +1 Loops over necessary minutes in an hour **(Points Earned)**
- ☐ +1 Calls isMinuteFree with period and another int parameter **(Points Earned)**
- ☐ +1 Keeps track of contiguous free minutes in a block (*algorithm*) **(Points Earned)**
- ☐ +1 Checks whether a valid block of duration minutes has been found **(Points Earned)**
- ☐ +1 Calculates and returns starting minute and −1 appropriately based on identified block (*algorithm*) **(Points Earned)**

**Canonical Solution:**

```
public int findFreeBlock(int period, int duration)
{
   int blockLength = 0;

   for (int minute = 0; minute < 60; minute++)
   {
      if (isMinuteFree(period, minute))
      {
         blockLength++;
         if (blockLength == duration)
         {
            return minute - blockLength + 1;
         }
      }
      else
      {
         blockLength = 0;
      }
   }
   return -1;
}
```

**Part B – makeAppointment (4 points)**

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned and −1 indicates a general or question-specific penalty.

+1 **[Skill 3.C]** Loops over periods from startPeriod through endPeriod (*no bounds errors*)

+1 **[Skill 3.A]** Calls findFreeBlock and reserveBlock with correct number of int parameters, representing a period and minute as appropriate, and duration

Responses can still earn the point even if they

- use incorrect parameter values

Responses will not earn the point if they

**FINAL Unit1-Unit2 FRQ - Topic order**

- use incorrect parameter types
- order the parameters incorrectly
- call the methods on the class or on an object other than this (use of this is optional)

+1 **[Skill 3.C]** Guards call to method to reserve a block by determining that starting minute is not −1

+1 **[Skill 3.C]** Books correct appointment and returns appropriate boolean (*algorithm)*

Responses can still earn the point even if they

- have incorrect bounds in the loop
- call findFreeBlock or reserveBlock incorrectly

Responses will not earn the point if they

- fail to return true or false return before the call to reserveBlock

**Canonical Solution:**

```
public boolean makeAppointment(int startPeriod,
                               int endPeriod,
                               int duration)
{
   for (int period = startPeriod;
        period <= endPeriod;
        period++)
   {
      int minute = findFreeBlock(period, duration);
      if (minute != -1)
      {
         reserveBlock(period, minute, duration);
         return true;
      }
   }
   return false;
}
```

| 0 | 1 | 2 | 3 | 4 ✓ |
|---|---|---|---|---|

Total number of points earned (minus penalties) is equal to 4.

☐ +1 Loops over periods from startPeriod through endPeriod (*no bounds errors*) **(Points Earned)**

## FINAL Unit1-Unit2 FRQ - Topic order

- ☐ +1 Calls findFreeBlock and reserveBlock with correct number of int parameters, representing a period and minute as appropriate, and duration **(Points Earned)**
- ☐ +1 Guards call to method to reserve a block by determining that starting minute is not −1 **(Points Earned)**
- ☐ +1 Books correct appointment and returns appropriate boolean (*algorithm)* **(Points Earned)**

**Canonical Solution:**

```
public boolean makeAppointment(int startPeriod,
                               int endPeriod,
                               int duration)
{
   for (int period = startPeriod;
        period <= endPeriod;
        period++)
   {
      int minute = findFreeBlock(period, duration);
      if (minute != -1)
      {
         reserveBlock(period, minute, duration);
         return true;
      }
   }
   return false;
}
```